Clases y estructuras

class y struct cambian los accesos predeterminados

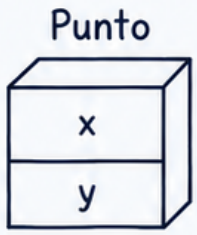
C++20 • Las dos formas pueden tener métodos, constructores y herencia.

Un tipo con datos públicos

`struct` suele usarse para agrupar datos con acceso público por defecto.

```
struct Punto
{
    double x{};
    double y{};
};


Punto p{3.0, 5.0};
```



Un tipo con interfaz pública

`class` suele expresar una interfaz que mantiene invariantes, con miembros privados por defecto.

```
class Alumno
{
    std::string nombre;
public:
    explicit Alumno(std::string n)
        : nombre(std::move(n)) {}

    const std::string& verNombre() const
    {
        return nombre;
    }
};
```

Compara los valores por defecto

Característica	class	struct
Acceso a miembros	private	public
Acceso de herencia	private	public
Constructores y destructores	Sí	Sí
Métodos y funciones virtuales	Sí	Sí
Miembros private y protected	Sí	Sí

Elige según el diseño

`struct` suele comunicar datos públicos relacionados. `class` suele expresar una interfaz que mantiene invariantes. Son convenciones de diseño; ambas palabras permiten controlar el acceso.

Evita esta confusión

Usar `struct` no garantiza compatibilidad con C. Depende de los tipos, la disposición de memoria y la interfaz binaria.

C++20: [class.access], [class.access.base]. C++ Core Guidelines C.2 y C.8.

Agregados y constructores

Las llaves pueden inicializar un agregado o llamar un constructor

C++20 • La definición del tipo determina qué ocurre.

1. Inicialización de agregado

Los valores inicializan `x` y `y` en su orden de declaración. Este tipo es un agregado.

```
struct Punto
{
    double x;
    double y;
};

Punto p{3.0, 5.0};
```

2. Llamada a un constructor

Las llaves seleccionan el constructor. Ese constructor inicializa los miembros mediante su lista.

```
struct PuntoConConstructor
{
    double x;
    double y;
    PuntoConConstructor(double a, double b)
        : x(a), y(b) {}
};

PuntoConConstructor q{3.0, 5.0};
```

¿Cuándo es un agregado?

Para una clase en C++20:

- no hay constructores declarados por el usuario ni heredados;
- no hay datos de instancia directos `private` o `protected`;
- no hay funciones virtuales;
- las bases no son virtuales, `private` ni `protected`.

Una distinción útil

Que un `struct` no declare constructor no basta para llamarlo agregado. Una `class` con miembros `public` también puede ser un agregado.

Compruébalo en código

```
std::is_aggregate_v<Punto>                  // true
std::is_aggregate_v<PuntoConConstructor>    // false
```

C++20: [dcl.init.aggr], [dcl.init.list].

Acceso a miembros y this

this apunta al objeto que ejecuta el método

C++20 • El acceso depende del miembro, no de escribir class o struct.

Acceso directo a un miembro público

Si un miembro es público, se puede acceder directamente desde fuera del objeto.

```
struct Registro
{
    std::string nombre;
};

Registro r{"Ana"};
r.nombre = "Luis";
```

Desde fuera del objeto

Los métodos públicos se usan para acceder y modificar los datos de un objeto.

```
Estudiante e{"Ana"};
e.fijarNombre("Luis");
// e.nombre = "Luis";  // Error: privado.
```

Dos formas equivalentes

Dentro de un método, se puede acceder a los miembros del objeto actual de estas dos formas equivalentes:

```
this->nombre
(*this).nombre
```

Interfaz de una clase

En una clase, los miembros pueden ser privados. Se usan métodos públicos para acceder y modificar los datos.

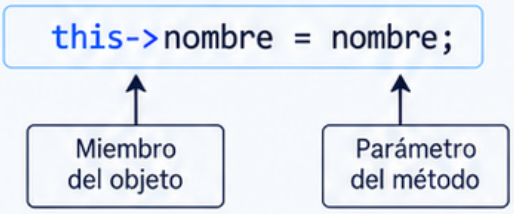
```
class Estudiante
{
    std::string nombre;
public:
    explicit Estudiante(std::string n)
        : nombre(std::move(n)) {}

    void fijarNombre(std::string nombre) {
        this->nombre = std::move(nombre);
    }

    const std::string& verNombre() const {
        return nombre;
    }
};
```

Lee la asignación

En `this->nombre = nombre`, la izquierda es el miembro y la derecha es el parámetro. `this->` evita el conflicto de nombres. Si no hay conflicto, normalmente es opcional.



Recuerda

`this` existe en métodos no estáticos. En un método `const` apunta a un objeto `const`. Los métodos públicos expresan operaciones del tipo; no es obligatorio crear un getter y un setter por cada dato.

C++20: [class.this], [class.access].

private, protected y public

El acceso cambia según el lugar desde donde se usa el miembro

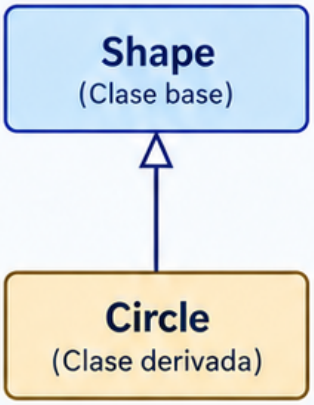
C++20 • Ejemplo con Circle : public Shape, sin declaraciones friend.

Clase base: Shape

```
class Shape
{
private:
    unsigned id{};

protected:
    std::string nombre;

public:
    void mostrar() const;
};
```



Clase derivada: Circle

```
class Circle : public Shape
{
public:
  void renombrar() {
    nombre = "Círculo"; // Permitido.
    // id = 1;          // Error.
  }
};
```

Acceso a los miembros de Shape

Desde...	private: id	protected: nombre	public: mostrar()
Un método de Shape	Sí	Sí	Sí
Un método de Circle, sobre `*this`	No	Sí	Sí
Código externo, como `main()`	No	No	Sí

Una precisión sobre protected

Dentro de `Circle`, acceder al miembro no estático protegido exige un objeto `Circle` o derivado de `Circle`. Una referencia `Shape&` no permite ese acceso por sí sola.

No confundas acceso con existencia

Los datos privados de Shapee forman parte del subobjeto base de `Circle`, aunque `Circle` no pueda nombrarlos directamente.

Alcance de la tabla

La herencia es `public`. Una base `private` o `protected` cambia el acceso heredado. Los miembros y amigos autorizados tienen reglas de acceso adicionales.

C++20: [class.access], [class.protected], [class.access.base].

Lista de inicialización

Inicializa los miembros antes de ejecutar el cuerpo

C++20 • La lista construye subobjetos; el cuerpo opera sobre ellos.

Declaraciones relevantes

La clase `Circle` tiene un miembro de datos, un miembro protegido y un constructor.

```
class Circle : public Shape
{
private:
    double radio;

protected:
    double factorEscala;

public:
    Circle(unsigned int id, double r);
};
```

Lista de inicialización

La lista construye la base y los miembros antes de ejecutar el cuerpo del constructor.

```
Circle::Circle(unsigned id, double r)
    : Shape(id, "Círculo"),
      radio(r > 0.0 ? r : 1.0),
      factorEscala(1.0)
{
    // Aquí empieza el cuerpo.
}
```

Orden real en este ejemplo

Las bases se construyen antes que los miembros. Los miembros siguen su orden de declaración, aunque cambies el orden escrito en la lista.

1. `Shape(id, "Círculo")`
2. `radio`
3. `factorEscala`
4. `Cuerpo del constructor`

Asignar en el cuerpo es distinto

Aquí los `double` sin inicializador previo reciben su valor mediante asignación. No deben leerse antes de ella. Un miembro `string` o `vector` ya habría sido construido antes de entrar al cuerpo.

```
Circle::Circle(unsigned id, double r)
    : Shape(id, "Círculo")
{
    radio = r > 0.0 ? r : 1.0;
    factorEscala = 1.0;
}
```

Cuándo hace falta inicializar

Las referencias deben enlazarse al construirse. Los miembros `const` necesitan inicialización y pueden tener un inicializador en su declaración. Para pasar argumentos al constructor de la base, usa la lista.

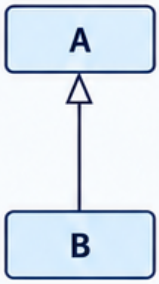
Tipos de herencia

Cuenta las bases directas y distingue los niveles

C++20 • En estos diagramas, la flecha apunta desde la derivada hacia su base.

1 Herencia simple

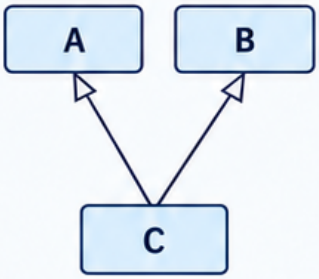
Una base directa.




```
class B : public A
{};
```

2 Herencia múltiple

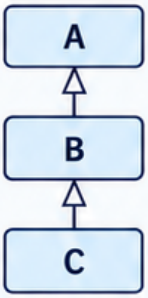
Dos o más bases directas.




```
class C : public A,
  public B {};
```

3 Herencia multinivel

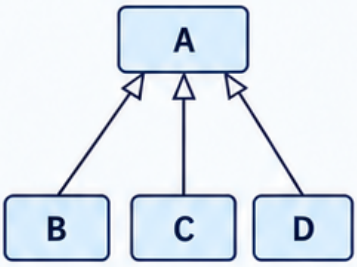
Una derivada también actúa como base.




```
class B : public A
{};
class C : public B
{};
```

4 Herencia jerárquica

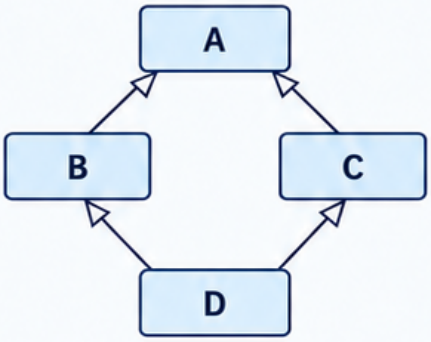
Varias derivadas comparten una base.




```
class B : public A {};
class C : public A {};
class D : public A {};
```

5 Herencia híbrida

Combina patrones de herencia. Un diamante combina varios niveles y bases múltiples.

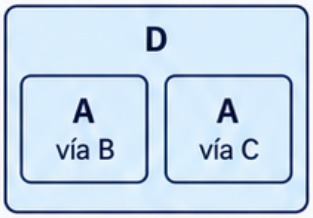



```
class A { /* ... */ };
class B : public A { /* ... */ };
class C : public A { /* ... */ };
class D : public B, public C { /* ... */ };
```

6 Diamante y base virtual

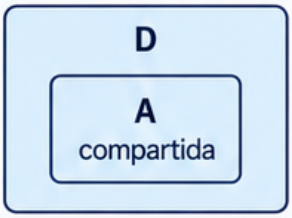
Sin `virtual`, D contiene dos subobjetos A: uno por cada rama. Con `virtual` en las dos ramas, comparte un solo A.

Sin base virtual



2 subobjetos A

Con base virtual



1 subobjeto A

```
struct A { int id{}; };
struct B : virtual public A {};
struct C : virtual public A {};
struct D : public B, public C {};
```

7 No son categorías excluyentes

La forma de la jerarquía y el acceso de herencia son decisiones distintas.
`public`, `protected` y `private` no significan simple o múltiple.
La base virtual la inicializa la clase más derivada.

C++20: [class.derived], [class.mi], [class.base.init].

Funciones virtuales y override

Una llamada virtual usa el tipo real del objeto

C++20 • Polimorfismo a través de un puntero o una referencia a la base.

Una interfaz abstracta

La clase base declara una función virtual pura. No se puede instanciar.

```
class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};
```

Una implementación concreta

La clase derivada implementa la función virtual y usa override.

```
class Circle : public Shape {
    double radio;
public:
    explicit Circle(double r) : radio(r) {}
    double area() const override {
        return std::numbers::pi * radio * radio;
    }
};
```

Sigue la llamada

Se hace la llamada a través de un puntero a la base. Se ejecuta la función del tipo real del objeto.

```
Circle c{5};
Shape* figura = &c;
std::cout << std::fixed
          << std::setprecision(2)
          << figura->area();
```

¿Qué ocurre?

La llamada virtual usa el tipo real del objeto apuntado.



Salida (área con dos decimales):

```
78.54
```

Qué aporta cada palabra

`virtual`	habilita el despacho dinámico.
`= 0`	declara una función virtual pura.
`Shape`	es abstracta y no se puede instanciar.
`override`	exige que la función redefina una virtual compatible de la base.

La firma importa

Si omites `const` en `Circle::area()`, no redefinirá esa función `const` de Shape. Con `override`, el compilador detecta el error.

```
class Circle : public Shape {
public:
    double area() override {  // Error: falta const
        return 0.0;
    }
};
```

Destrucción polimórfica

Si eliminas un objeto derivado mediante un puntero a la base, la base necesita un destructor virtual.
Pasar o copiar un derivado por valor como Shape puede perder su parte derivada cuando la base es concreta: ese problema se llama *slicing*.

C++20: [class.virtual], [class.abstract], [expr.delete].

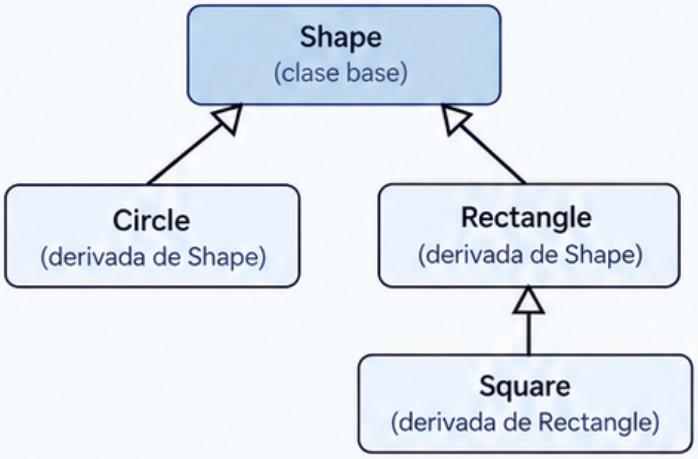
Polimorfismo con Shape

El mismo recorrido calcula áreas de objetos distintos

C++20 • Shape, Circle, Rectangle y Square.

La jerarquía del ejemplo

Square tiene una base directa: **Rectangle**. La cadena hasta **Shape** es herencia multinivel, no múltiple.



Un solo recorrido

```cpp
Circle c{5.0};
Rectangle r{4.0, 6.0};
Square s{3.0};

Shape* figuras[]{&c, &r, &s};

for (const Shape* figura : figuras) {
    std::cout << figura->area() << '\n';
}
```

Salida con dos decimales

```
78.54
24.00
9.00
```

Orden de salida: Circle, Rectangle, Square.
El programa completo fija la precisión con `std::fixed` y `std::setprecision(2)`.

Todas comparten la operación

Shape declara:

```cpp
virtual double area() const = 0;
virtual ~Shape() = default;
```

Circle redefine: pi × radio × radio.
Rectangle redefine: base × altura.
Square redefine: lado × lado.
Las tres implementaciones usan `const override`.

Cálculo de área en cada clase

`Circle::area()` : pi × radio × radio

`Rectangle::area()` : base × altura

`Square::area()` : lado × lado

Fragmentos didácticos: las clases completas están en `ejemplos/`.

Durante la construcción

Una llamada virtual desde un constructor o destructor se resuelve en la clase que está construyéndose o destruyéndose; no salta a una parte derivada que todavía no está activa.

Límites del modelo

Esta jerarquía es didáctica. Si Rectangle permite cambiar base y altura por separado, derivar Square puede romper ese contrato. Revisa los invariantes antes de reutilizar esta relación en otro diseño.

C++20: [class.virtual], [class.cdtor]. C++ Core Guidelines C.120.